

AArch64 Arithmetic, Logic, Shifts, and Flags

Integer arithmetic, logical operations, shifts, aliases, condition flags, and machine-level interpretation

Computer Organization & Architecture | AArch64 Reference

Topic 5 Outline

Topic Objective

Understand how AArch64 data-processing instructions operate on fixed-width bit patterns and how NZCV flags support signed and unsigned interpretation.

- **Section 1:** Fixed-width arithmetic and interpretation
- **Section 2:** Flag-setting arithmetic and comparisons
- **Section 3:** Logical operations and bit masks
- **Section 4:** Shifts, rotates, immediates, and aliases
- **Section 5:** Summary and self-test

Fixed-Width Arithmetic and Interpretation

Arithmetic Operates on Bit Patterns

Architectural View

AArch64 integer arithmetic operates on fixed-width register contents. The instruction computes a bit-pattern result; signedness is an interpretation applied by software and branch conditions.

Same Operation

add x0, x1, x2

- Adds two 64-bit register values.
- Produces one 64-bit result.
- Does not by itself mean signed or unsigned.

Different Interpretations

- Unsigned: range is 0 to $2^{64} - 1$.
- Signed: two's complement range is -2^{63} to $2^{63} - 1$.
- Flags help interpret exceptional cases.

Add and Subtract

Addition

`add x0, x1, x2`

- Computes $x1 + x2$.
- Writes the result to `x0`.
- Does not update NZCV flags.

Subtraction

`sub x0, x1, x2`

- Computes $x1 - x2$.
- Uses fixed-width modular arithmetic.
- Does not update NZCV flags.

Key Point

The same binary datapath can support signed and unsigned arithmetic. Interpretation comes from how the result and flags are used.

Subtraction Uses Two's Complement

Hardware Idea

Subtraction can be implemented as addition of the two's-complement negation.

$$A - B = A + (\sim B) + 1$$

Architectural Example

```
sub x0, x1, x2
```

- Reads x1 and x2.
- Produces $x1 - x2$.
- Result is fixed-width.

Pedagogical Warning

- Negative results wrap modulo 2^n .
- The same bits can be read as signed or unsigned.
- Flags are needed to reason about overflow and borrow.

Flag-Setting Arithmetic and Comparisons

NZCV Flags



- **N**: result sign bit is 1.
- **Z**: result is exactly zero.
- **C**: unsigned carry-out for addition; no-borrow for subtraction.
- **V**: signed two's-complement overflow.

Interpretation Rule

C is mainly useful for unsigned reasoning; V is mainly useful for signed reasoning.

Flag-Setting Arithmetic

No Flag Update

`add x0, x1, x2`

`sub x0, x1, x2`

- Write a register result.
- Leave NZCV unchanged.

Flag Update

`adds x0, x1, x2`

`subs x0, x1, x2`

- Write a register result.
- Update NZCV from that result.

Important Limitation

Do not overgeneralize this as “add s to every mnemonic.” AArch64 has specific flag-setting instruction forms and aliases.

Carry and Overflow Are Different

One Operation, Two Interpretations

The same addition can be valid in one interpretation and overflow in another.

View	Example	Meaning
Unsigned	$255 + 1 = 0$ in 8 bits	Carry-out indicates unsigned wraparound.
Signed	$127 + 1 = -128$ in 8 bits	Overflow indicates signed range exceeded.

AArch64 Consequence

Unsigned branches inspect C; signed branches inspect combinations involving N and V.

Carry Means No-Borrow After Subtraction

AArch64 Subtraction Rule

After subs or cmp, C=1 means the subtraction did **not** require an unsigned borrow.

Operation	Unsigned Relation	Carry Flag
5 - 3	$5 \geq 3$	C=1: no borrow
3 - 5	$3 < 5$	C=0: borrow needed

Common Mistake

Do not read C=1 after subtraction as “borrow occurred.” In AArch64, it means the opposite: **no borrow**.

cmp Is an Alias

Comparison Without Keeping the Difference

cmp performs a subtraction for flags, but discards the subtraction result.

```
cmp x0, x1  
subs xzr, x0, x1
```

- The two lines express the same underlying operation.
- $x0 - x1$ is used to update NZCV.
- The result is written to xzr, so it is discarded.

Signed and Unsigned Branches

After a Compare

The same `cmp` can support signed or unsigned comparisons depending on which conditional branch is used.

Branch	Interpretation	Condition
<code>b.lt label</code>	signed less than	$N \neq V$
<code>b.ge label</code>	signed greater/equal	$N = V$
<code>b.lo label</code>	unsigned lower	$C = 0$
<code>b.hs label</code>	unsigned higher/same	$C = 1$

`cmp x0, x1      b.lt signed_less      b.lo unsigned_lower`

Worked Comparison Example

Same Bits, Different Ordering

Consider 8-bit values for intuition: `0xff` and `0x01`.

Interpretation	Values	Result
Unsigned	255 and 1	$255 > 1$
Signed two's complement	-1 and 1	$-1 < 1$

Teaching Point

The bit patterns are unchanged. The branch mnemonic determines whether the flags are interpreted as signed or unsigned evidence.

Logical Operations and Bit Masks

Logical Instructions Operate Bit-by-Bit

Bitwise Meaning

Logical instructions operate independently on corresponding bit positions. They do not treat the register as a signed or unsigned number.

`and x0, x1, x2` `orr x0, x1, x2` `eor x0, x1, x2`

- `and`: keeps bits set in both operands.
- `orr`: sets bits present in either operand.
- `eor`: sets bits that differ between operands.

Bitwise AND Example

Masking

A mask selects which bit positions should survive an operation.

```
  11001010
& 10110100
-----
  10000000
```

AArch64 Example

```
and x0, x1, x2
```

The result contains only the bit positions that were set in both source registers.

tst Is an Alias

Testing Bits Without Keeping the Result

tst computes a bitwise AND for flags, but discards the AND result.

```
tst x0, x1  
ands xzr, x0, x1
```

- The AND result updates NZCV.
- The result is written to xzr, so it is discarded.
- This is useful for asking whether selected bits are zero or nonzero.

Logical Operations and Flags

Plain Logical Operations

`and x0, x1, x2`

`orr x0, x1, x2`

`eor x0, x1, x2`

- Write a register result.
- Do not update NZCV.

Flag-Setting Form

`ands x0, x1, x2`

`tst x1, x2`

- Updates flags from the logical result.
- `tst` discards the result.

Avoid Overgeneralization

There is no universal “add s to every logical mnemonic” rule. Teach the actual AArch64 forms students will use.

Shifts, Rotates, Immediates, and Aliases

Shift Operations

Shift	Meaning	Typical Interpretation
<code>lsl #n</code>	shift left, zeros enter right	multiply by 2^n if overflow is ignored
<code>lsr #n</code>	logical right, zeros enter left	unsigned division by powers of two
<code>asr #n</code>	arithmetic right, sign bit is replicated	preserves signed negative values
<code>ror #n</code>	rotate right, bits wrap around	bit manipulation, not division

Key Distinction

`lsr` and `asr` differ when the top bit is set. That difference matters for unsigned versus signed interpretation.

Shift Example

Logical Shift Right

```
lsr x0, x1, #1
```

- Zero enters the top bit.
- Good for unsigned bit patterns.
- Does not preserve a negative sign.

Arithmetic Shift Right

```
asr x0, x1, #1
```

- Original sign bit is copied.
- Preserves signed interpretation.
- Useful for signed right shifts.

Qualification

Shifts can resemble multiplication or division by powers of two, but finite width, overflow, and signed rounding behavior must be considered.

Immediate Operands Are Encoded in the Instruction

Immediate Operand

An immediate is a constant stored directly inside the instruction encoding.

```
add x0, x1, #5
and x2, x3, #0xff
```

- Immediate operands avoid loading a separate constant from memory.
- They are limited by the available instruction bits.
- AArch64 immediate forms are constrained by instruction class.

Preparation for Encoding

Do not assume `add x0, x1, #anything` can encode any 64-bit constant. The exact rules are covered later with instruction encoding.

mov Is Often an Alias

Assembler Convenience

In AArch64, `mov` is frequently assembler syntax that maps to different underlying A64 instructions depending on the operands.

```
mov x0, x1
```

```
mov x0, #5
```

```
mov x0, xzr
```

- Students should read `mov` as “put this value there.”
- The assembler chooses an appropriate encoding.
- The exact expansion is an encoding topic, not a semantic mystery.

Multiplication Is Data Processing, Not Necessarily Same Latency

Architectural Category

Multiplication is a data-processing operation: it reads registers and writes a register result.

```
mul x0, x1, x2
```

- Architecturally, it belongs with register-based computation.
- Microarchitecturally, it may use a separate multiply unit.
- Its latency can differ from simple add/sub/logic instructions.

No Fixed Cycle Claim

AArch64 specifies the instruction's architectural effect, not a universal number of cycles.

Short Worked Program

Goal

Load a value, test whether it is zero, and add 1 only if it is nonzero.

```
ldr x1, [x0]
cmp x1, xzr
b.eq done
add x1, x1, #1
str x1, [x0]
done:
```

Mechanism

cmp updates flags using a discarded subtraction. b.eq branches if the zero flag is set.

Summary and Self-Test

Topic 5 Summary

- AArch64 integer arithmetic operates on fixed-width bit patterns.
- `add` and `sub` do not inherently mean signed or unsigned arithmetic.
- C supports unsigned reasoning; V supports signed overflow reasoning.
- After subtraction, `C=1` means no unsigned borrow occurred.
- `cmp` and `tst` are aliases that update flags and discard the computed result.
- Shifts are useful arithmetic tools, but overflow and signed interpretation matter.

Self-Test 1/2

- ① Why do add and sub not by themselves determine signedness?
- ② What is the difference between the C and V flags?
- ③ After `cmp x0, x1`, what does `C=1` mean for unsigned comparison?
- ④ Explain why `cmp x0, x1` can be understood as `subs xzr, x0, x1`.
- ⑤ Which branch should be used for signed less-than: `b.lt` or `b.lo`?

Self-Test 2/2

- 6 What does `tst x0, x1` test, and why is the result discarded?
- 7 Why is `asr` different from `lsr` when the sign bit is set?
- 8 Why can immediate operands not represent every possible 64-bit constant?
- 9 Why should `mov` be treated as an assembler convenience rather than one single universal hardware operation?
- 10 Why is it unsafe to claim that every AArch64 instruction has a fixed cycle count?

Topic 5 Summary

Core Message

AArch64 arithmetic and logical instructions operate on fixed-width bit patterns; interpretation depends on the instruction form, the flags, and the surrounding program context.

- Arithmetic instructions define architectural state changes.
- Flag-setting forms connect arithmetic results to conditional control flow.
- Aliases simplify assembly syntax without changing the underlying operation.
- Shifts and logical operations manipulate bit patterns directly.
- Microarchitectural performance depends on implementation, not only the ISA.